

Claude Code en pratique : trouver les bons outils

Cheat sheet opérationnelle - Formation à l'IA agentique

Pôle Formation UIMM-CVDL - S. JAUBERT - Document de référence à conserver près du poste de travail

À qui s'adresse ce document. À toute personne qui commence à utiliser Claude Code (l'assistant IA qui travaille dans un terminal, c'est-à-dire la fenêtre où l'on tape des commandes texte). Aucune compétence informatique avancée n'est requise : chaque astuce est expliquée en langage clair, avec la commande exacte à recopier et un cas d'usage concret.

Comment lire les commandes. Tout ce qui apparaît sur **fond sombre** se tape (ou se copie-colle) dans le terminal, puis on valide avec la touche Entrée. Le symbole `|` (barre verticale) et le symbole `>` (chevron) sont expliqués dans le glossaire en fin de document.

Partie 1 - Trouver le bon outil sans le connaître à l'avance

Claude Code peut être enrichi par des **skills** (des modules de compétences spécialisés) et des **connecteurs MCP** (des ponts vers vos logiciels : messagerie, CRM, gestion de projet, etc.). Le problème n'est pas qu'ils manquent : c'est qu'on ne sait pas lequel installer, ni si on possède déjà l'équivalent. D'où une perte de temps fréquente.

A

Installer le seul outil qui trouve tous les autres : `find-skills`

Une seule installation rend l'ensemble du catalogue consultable. On décrit ensuite ce que l'on veut faire **en français courant**, et l'outil propose les modules les plus pertinents, classés par fiabilité.

```
npx skills add vercel-labs/skills@find-skills
```

La bonne façon de formuler sa demande : décrire le résultat voulu, pas le nom d'un outil.

À éviter

"trouve-moi un skill Apollo"

Recommandé

"j'ai besoin de rechercher des décideurs dans des entreprises financées, par secteur et par fonction"

Cas d'usage formation/industrie : "trouve un module qui transforme un compte rendu de réunion en liste d'actions", ou "je veux extraire les tableaux d'un PDF de spécifications". Plus l'intention est précise, plus la réponse est utile.

B

Lire le nombre d'installations avant d'installer

Le catalogue est ouvert : n'importe qui peut publier. Le seul signal honnête de qualité est le nombre de personnes qui ont installé l'outil et l'ont gardé.

Nombre d'installations	Interprétation
Moins de 50	Peu éprouvé. Vous êtes proche du statut de testeur. Vérifiez le dépôt avant d'installer.
Plus de 100	Des personnes autres que l'auteur l'ont trouvé assez utile pour l'adopter. Signal plus rassurant.

Réflexe de vérification avant d'installer un module peu téléchargé : regarder la date du dernier commit (dernière mise à jour). S'il n'a pas bougé depuis six mois, sans activité ni étoiles, la réponse est non.

Le réflexe à retenir. Avant d'installer quoi que ce soit, demandez à Claude : "ai-je déjà de quoi faire cela ?". Bien souvent un connecteur déjà en place fait le travail, et une installation supplémentaire est inutile. `find-skills` vérifie justement ce que vous possédez déjà avant de proposer du neuf.

Partie 2 - Les 13 astuces qui font la différence dans le terminal

Toutes ces astuces reposent sur des principes simples et anciens (les "tuyaux" entre logiciels, la redirection vers un fichier, les raccourcis). Combinées à Claude, elles accélèrent fortement le travail. Pour chaque astuce : la façon débutante, la façon pro, et un cas concret.

1

Lancer une commande sans quitter Claude : le `!`

Débutant

Quitte Claude, ouvre un autre terminal, puis revient expliquer ce qui s'est passé.

Pro

Tape `!` suivi de la commande, sans quitter la session.

```
! npm run build
! git status
```

Le résultat reste sous les yeux de Claude, qui peut réagir immédiatement. On ne perd jamais le fil de la conversation.

Cas d'usage : vérifier l'état d'un dossier ou relancer une opération sans interrompre l'échange en cours.

2

Envoyer une erreur directement à Claude

Débutant

Copie le message d'erreur, le colle dans Claude et écrit "corrige ça".

Pro

Envoie la sortie d'erreur d'un seul geste.

```
npm run build 2>&1 | claude -p "corrige ces erreurs"
```

Claude lit le message exact (et non votre reformulation), avec les numéros de ligne. C'est plus rapide et plus précis.

Cas d'usage : un programme échoue avec un message incompréhensible : on le transmet tel quel à Claude pour diagnostic.

3

Mode rapide (sans conversation) : `claude -p`

Débutant

Ouvre une session complète même pour une question d'une ligne.

Pro

Utilise `-p` pour une réponse unique et immédiate.

```
claude -p "explique ce que fait ce code"
claude -p "génère un fichier .gitignore pour un projet Python"
```

Le drapeau `-p` (pour "print", afficher) traite la demande, affiche la réponse et s'arrête. Idéal pour une explication ou une recherche ponctuelle.

Cas d'usage : obtenir une définition, un exemple ou un petit fichier sans lancer tout un échange.

4

Reprendre une session : `--resume`

Débutant

Recommence de zéro et ré-explique tout le contexte à chaque fois.

Pro

Reprend la veille exactement là où il s'était arrêté.

```
claude --resume
```

Recharge la dernière conversation : le contexte, les décisions et les fichiers sont toujours là.

Cas d'usage : un dossier traité sur plusieurs jours. Le lendemain, Claude se souvient de ce qui a été tenté et de ce qui a fonctionné.

5

Enchaîner Claude avec d'autres outils

Débutant

Lance chaque outil séparément et recopie le résultat de l'un dans l'autre.

Pro

Relie les outils en une seule commande.

```
git diff | claude -p "relis ces modifications"
cat agenda.txt | claude -p "résume les points clés"
```

Le symbole `|` (le "tuyau") fait passer le résultat d'un outil dans Claude. Tout ce qui produit du texte peut être analysé.

Cas d'usage : journaux de bord, sorties de tests, fichiers de configuration, exports... tout texte devient analysable.

6

Créer des raccourcis (alias) pour les tâches répétitives

Débutant

Retape la commande complète à chaque fois.

Pro

Crée un raccourci de deux lettres.

```
# À ajouter une fois dans le fichier ~/.bashrc
alias cr='git diff | claude -p "relis ce code"'
alias cs='claude -p "résume ce projet en 3 phrases"'
```

Après rechargement, taper `cr` suffit. Les gains de temps se cumulent sur des centaines d'usages.

Cas d'usage : automatiser un contrôle ou un résumé que l'on déclenche plusieurs fois par jour.

7

Donner les vraies données Git à Claude

Débutant

Demande "qu'est-ce que j'ai changé ?" et attend.

Pro

Fournit directement l'historique réel.

```
git log --oneline -10 | claude -p "résume ces changements"
```

Claude travaille sur les données exactes, sans deviner à partir des dates de fichiers.

Cas d'usage : produire un compte rendu fidèle des évolutions récentes d'un projet versionné.

8

Intégrer Claude dans des scripts automatisés

Débutant

Ne sait pas comment utiliser Claude en automatique.

Pro

Utilise `--print` dans des scripts ou des tâches planifiées.

```
# Contrôle quotidien planifié (exemple)
claude -p "fais un bilan rapide de ce projet" > bilan.txt
```

En mode `--print`, Claude se comporte comme n'importe quel outil : il reçoit une entrée, produit un texte. On peut donc l'inclure dans une tâche récurrente.

Cas d'usage : un bilan automatique chaque matin, écrit dans un fichier, sans intervention.

9

Lancer une tâche longue en arrière-plan : le `!`

Débutant

Ouvre un autre onglet pour une commande qui tourne longtemps.

Pro

Lance la commande avec `!` et continue à discuter.

```
! npm run test -- --watch
```

La commande tourne en fond pendant que l'on poursuit l'échange. Claude peut s'y référer ensuite.

Cas d'usage : un serveur de test qui se recharge tout seul pendant que Claude apporte des modifications.

10

Mémoriser ses réglages une bonne fois pour toutes

Débutant

Reconfigure sa clé d'accès à chaque nouveau terminal.

Pro

L'enregistre une fois dans son profil et n'y pense plus.

```
# À ajouter une fois dans ~/.bashrc
alias cc='claude'
```

Un raccourci de deux lettres économise des milliers de frappes sur un mois.

Cas d'usage : lancer Claude avec `cc` au lieu de `claude` à chaque fois.

11

Donner plusieurs instructions en une seule fois

Débutant

Envoie une instruction, attend, puis la suivante.

Pro

Enchaîne la séquence complète dans une seule demande.

```
claude -p "lance les tests, corrige ceux qui échouent, puis relance jusqu'à ce que tout passe"
```

Claude exécute la suite de manière autonome : chaque étape nourrit la suivante.

Cas d'usage : déléguer une procédure en plusieurs étapes plutôt que de la piloter pas à pas.

12

Enregistrer la réponse de Claude dans un fichier

Débutant

Copie la réponse et la colle manuellement dans un fichier.

Pro

Redirige la sortie vers un fichier d'un seul geste.

```
claude -p "rédige un mode d'emploi du projet" > LISEZMOI.md
```

Le chevron `>` écrit la réponse directement dans le fichier. Avec `>>`, on ajoute à la suite d'un fichier existant.

Cas d'usage : générer une note, un mode d'emploi ou un modèle de document prêt à l'emploi.

13

Faire lire les vrais fichiers à Claude : `cat`

Débutant

Décrit ses fichiers avec des mots.

Pro

Fournit les fichiers réels en entrée.

```
cat notes.txt compte-rendu.txt | claude
-p "synthétise ces documents"
```

Claude lit le contenu réel, sans interpréter une description. Le symbole `*` permet de transmettre tout un lot de fichiers d'un coup.

Cas d'usage : réunir plusieurs documents pour en faire une synthèse unique. **À noter :** pour de très gros volumes, préférer la session interactive, où Claude lit les fichiers de façon sélective.

Glossaire express - les symboles à connaître

Terminal

La fenêtre dans laquelle on tape des commandes texte plutôt que de cliquer. C'est là qu'on dialogue avec Claude Code.

! (point d'exclamation)

Placé devant une commande, il l'exécute sans quitter Claude (astuces 1 et 9).

| (barre verticale, "le tuyau")

Fait passer le résultat d'un outil dans un autre. Exemple : `git diff | claude -p "..."` envoie les modifications à Claude.

> et >> (chevrons)

`>` écrit le résultat dans un fichier (en remplaçant son contenu). `>>` ajoute à la fin sans effacer.

2>&1

Capture à la fois les messages normaux et les messages d'erreur, pour tout transmettre à Claude (astuce 2).

-p ou --print

Mode rapide : Claude répond une fois et s'arrête, sans ouvrir de conversation (astuces 3, 8, 12).

alias

Un raccourci personnel : on remplace une commande longue par quelques lettres (astuces 6 et 10).

cat

Commande qui affiche le contenu d'un fichier ; ici, elle sert à transmettre ce contenu à Claude (astuce 13).

skill / connecteur MCP

Un skill est un module de compétence ajouté à Claude. Un connecteur MCP est un pont vers un logiciel existant (messagerie, gestion de projet, CRM...).

Le principe à garder en tête. La différence entre un usage débutant et un usage avancé ne tient pas à des astuces d'IA, mais à quelques réflexes simples : relier les outils entre eux (le tuyau), enregistrer les résultats (le chevron), créer des raccourcis, et donner à Claude les vraies données plutôt qu'une description. Commencez par maîtriser deux ou trois de ces gestes, le reste viendra naturellement.